

ROLES & PERMISSIONS V3 — Architecture & Frontend

Integration Manual

Version: 3.0 (Simplified) **Date:** September 12, 2026 **Branch:** devG

PART 1: ARCHITECTURE

1.1 Overview

The RBAC system controls who can do what in the ERP. It has three actors:

1. **Super Admin (SA)** — Platform-level. Hardcoded bypass for all permissions. Sets what each firm's admin can do.
2. **Firm Admin** — Manages their law firm. Distributes permissions to employee roles within the ceiling SA set.
3. **Employees** — Have roles (ADVOCATE, PARALEGAL, CLIENT, or custom). Each role has a set of permissions.

The core rule: No employee ever holds a permission their Firm Admin doesn't hold. No Firm Admin ever holds a permission Super Admin hasn't allowed.

1.2 Data Model

SYSTEM ROLES (boot-seeded, global)
SUPER_ADMIN FIRM_ADMIN ADVOCATE PARALEGAL
firm_id = NULL, is_system = true
FIRM ROLES (per firm, cloned at onboarding)
FIRM_ADMIN ADVOCATE PARALEGAL CLIENT
firm_id = <firm>, is_system = false
CUSTOM ROLES (created by SA or Firm Admin)
Any code, firm_id = <firm>, is_system = false
PERMISSIONS (flat catalog)
code = "MODULE:ACTION", scope = TENANT GLOBAL
~120+ permissions across 16 modules

Key tables:

Table	Purpose
roles	System roles + firm roles + custom roles
permissions	All permission codes (MODULE:ACTION)
role_permissions	Junction: which role has which permissions
users	Has role_id FK — the source of truth for user's role

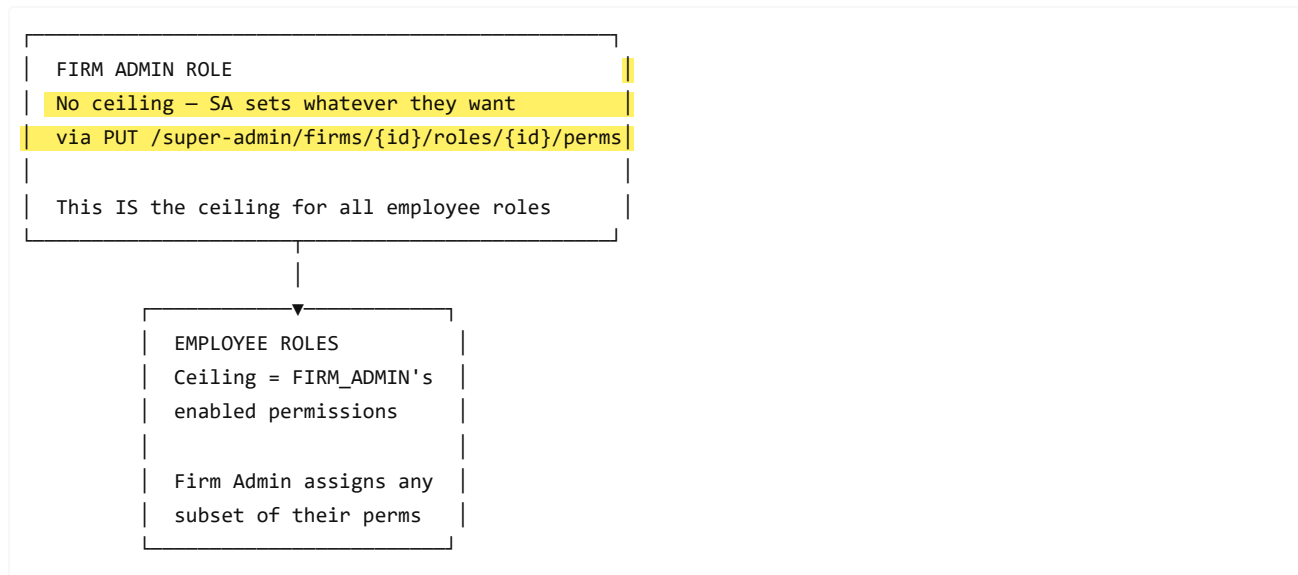
The Role entity is simple:

- id , roleName , roleCode , firmId (null = system), isSystem , applicableTo , description , active

No templates. No parentRoleId. No lastSaEditAt. Just roles.

1.3 The Ceiling Model

This is the most important concept to understand:



FIRM_ADMIN: No ceiling. SA can set any non-GLOBAL permission. This is the "bucket" of permissions available for the firm.

ADVOCATE, PARALEGAL, CLIENT, Custom roles: Ceiling = the firm's FIRM_ADMIN's currently enabled permissions. Firm Admin can only distribute what they have.

Example:

1. SA sets Firm A's FIRM_ADMIN: [CASE:VIEW, CASE:CREATE, BILLING:VIEW]
2. Firm Admin assigns ADVOCATE: [CASE:VIEW, CASE:CREATE] ✅ (within ceiling)
3. Firm Admin assigns PARALEGAL: [BILLING:VIEW] ✅ (within ceiling)
4. Firm Admin tries to assign ADVOCATE: [SCRAPER:VIEW] ❌ (not in ceiling — FIRM_ADMIN doesn't have it)

1.4 Permission Codes

Format: MODULE:ACTION

Modules (16 total): CASE_MANAGEMENT, DOCUMENT_MANAGEMENT, CLIENT_MANAGEMENT, BILLING, CALENDAR, EMPLOYEE, REPORTS, AUDIT, PROJECT_MANAGEMENT, ROLE_MANAGEMENT, FIRM_MANAGEMENT, MENU_MANAGEMENT, PERMISSION_MANAGEMENT, SCRAPER_MANAGEMENT, USER_MANAGEMENT, DASHBOARD_MANAGEMENT

Standard Actions (every module): ACCESS, VIEW, CREATE, EDIT, DELETE

Extra Actions (module-specific):

- CASE_MANAGEMENT: ASSIGN, ARCHIVE, UPDATE_STATUS
- DOCUMENT_MANAGEMENT: UPLOAD, DOWNLOAD, SHARE
- BILLING: APPROVE, EXPORT
- CALENDAR: SCHEDULE
- REPORTS: EXPORT, PRINT
- PROJECT_MANAGEMENT: CREDENTIAL_VIEW, CREDENTIAL_REVEAL
- SCRAPER_MANAGEMENT: EXPORT

Scope:

- TENANT — normal permission (can be assigned to roles)
 - GLOBAL — SA-only (never assignable to firm roles)
-

1.5 How Permissions Are Evaluated at Runtime

```
Request comes in → JwtAuthFilter extracts user
↓
PermissionEvaluator.require("CASE_MANAGEMENT:VIEW")
↓
Is user SuperAdmin? → YES → Allow (hardcoded bypass)
↓ NO
Check in-memory cache (5min TTL)
↓ Cache miss
Load from JWT claims (fast path) or DB fallback
↓
Is permission in user's set? → YES → Allow
↓ NO
Throw ForbiddenException (403)
```

Session invalidation: When permissions change, `permissionVersion` is bumped for affected users. JWT carries this version — mismatch forces re-login.

1.6 API Surface

Super Admin Endpoints

Method	Endpoint	Purpose
POST	/api/v1/super-admin/register	Register SA
POST	/api/v1/super-admin/login	Login SA
POST	/api/v1/super-admin/firms	Create firm (clones 4 roles)
GET	/api/v1/super-admin/firms/{firmId}/roles	View firm's roles + permissions
PUT	/api/v1/super-admin/firms/{firmId}/roles/{roleId}/permissions	Set role permissions (override)
POST	/api/v1/super-admin/firms/{firmId}/roles	Create custom role on-behalf
POST	/api/v1/admin/roles	Create/update system role
GET	/api/v1/admin/roles	List all system roles
POST	/api/v1/admin/roles/permissions	Assign permissions to system role

Firm Admin Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Login (with lawFirmCode)
GET	/api/v1/firm/roles	List firm roles + user counts
GET	/api/v1/firm/roles/{roleId}/permissions	View permissions + available ceiling
PUT	/api/v1/firm/roles/{roleId}/permissions	Update permissions (within ceiling)
POST	/api/v1/firm/roles	Create custom role
DELETE	/api/v1/firm/roles/{roleId}	Delete custom role
PATCH	/api/v1/firm/roles/{roleId}/toggle	Toggle role status

GET	/api/v1/firm/roles/{roleId}/users	List users in role
-----	-----------------------------------	--------------------

1.7 Request/Response Format

All requests use the wrapper format:

```
{
  "data": { ... }
}
```

All responses use:

```
{
  "success": true,
  "message": "...",
  "data": { ... }
}
```

1.8 Per-Firm Isolation

Firm A and Firm B are completely independent:

- Each firm has its own set of roles (FIRM_ADMIN, ADVOCATE, PARALEGAL, CLIENT, custom)
- Each firm's FIRM_ADMIN has its own permission set (set by SA)
- Changing Firm A's permissions has zero effect on Firm B
- The `roles` table has `firm_id` FK — all queries are scoped by firm

PART 2: FRONTEND INTEGRATION MANUAL

2.1 Authentication Flow

Super Admin Login

1. POST `/api/v1/super-admin/login`
Body: `{ "data": { "username": "...", "password": "..." } }`
2. Response: `{ "data": { "accessToken": "...", "refreshToken": "..." } }`
3. If MFA enabled: `{ "data": { "status": "MFA_REQUIRED", "mfaToken": "..." } }`
→ POST `/api/v1/super-admin/login` with `{ "data": { "username": "...", "password": "...", "totpCode": "123456" } }`
4. Store `accessToken` in memory (not `localStorage` for security)

Firm Admin Login

1. POST `/api/v1/auth/login`
Body: `{ "data": { "lawFirmCode": "TESTFIRM", "username": "...", "password": "..." } }`
2. Response may include:
 - `{ "data": { "accessToken": "...", "refreshToken": "..." } }` → Success
 - `{ "data": { "status": "PASSWORD_CHANGE_REQUIRED", "passwordChangeToken": "..." } }` → Must change password

```
- { "data": { "status": "MFA_REQUIRED", "mfaToken": "..." } } → MFA flow
```

3. For PASSWORD_CHANGE_REQUIRED:

```
POST /api/v1/auth/change-password
```

```
Body: { "data": { "token": "<passwordChangeToken>", "newPassword": "NewPass123!" } }
```

Then login again.

4. For MFA:

```
POST /api/v1/auth/mfa/validate
```

```
Body: { "data": { "mfaToken": "<mfaToken>", "totpCode": "123456" } }
```

Token Management

- Store accessToken in memory (React state / Vuex / Pinia)
- Store refreshToken in httpOnly cookie or secure storage
- Attach to every request: Authorization: Bearer <accessToken>
- On 401 → try refresh → if fails → redirect to login
- On "permissions changed" (401 with specific message) → force re-login

2.2 Super Admin Dashboard — Firm Management

Create a New Firm

1. POST /api/v1/super-admin/firms

Body: {

 "data": {

 "lawFirmCode": "NEWFIRM",

 "name": "New Law Firm",

 "firmType": "FIRM",

 "email": "info@newfirm.com",

 "phone": "9800000000",

 "adminUsername": "admin",

 "adminEmail": "admin@newfirm.com",

 "adminMobileNo": "9800000001",

 "adminPassword": "SecurePass123!",

 "adminFullName": "Firm Admin"

 }

}

2. Response: { "data": { "firmId": "...", "lawFirmCode": "NEWFIRM", "adminUsername": "admin" } }

3. Firm is created with 4 roles (FIRM_ADMIN, ADVOCATE, PARALEGAL, CLIENT) — all EMPTY (0 permissions)

Set Firm Admin Permissions (The Ceiling)

1. GET /api/v1/super-admin/firms/{firmId}/roles

→ Get the FIRM_ADMIN role ID

2. GET /api/v1/admin/roles

→ Get available permission IDs from the system FIRM_ADMIN role

3. PUT /api/v1/super-admin/firms/{firmId}/roles/{firmAdminRoleId}/permissions

Body: {

 "data": {

 "roleId": "<firmAdminRoleId>",

```
    "permissionIds": [
      "<CASE_MANAGEMENT_VIEW_ID>",
      "<CASE_MANAGEMENT_CREATE_ID>",
      "<BILLING_VIEW_ID>",
      ...
    ]
  }
}
```

4. This sets the ceiling. Firm Admin can now distribute these permissions.

Create Custom Role On-Behalf

1. POST /api/v1/super-admin/firms/{firmId}/roles
Body: { "data": { "name": "Senior Advocate", "code": "SENIOR_ADVOCATE" } }
2. PUT /api/v1/super-admin/firms/{firmId}/roles/{customRoleId}/permissions
Body: { "data": { "roleId": "...", "permissionIds": [...] } }
→ SA can assign ANY non-GLOBAL permission (no ceiling)

2.3 Firm Admin Dashboard — Role & Permission Management

View All Roles

GET /api/v1/firm/roles

```
Response: {
  "data": [
    {
      "id": "...",
      "name": "FIRM_ADMIN",
      "code": "FIRM_ADMIN",
      "userCount": 1,
      "assignedUserNames": ["Firm Admin User"],
      "isActive": true,
      "isSystem": false
    },
    {
      "id": "...",
      "name": "ADVOCATE",
      "code": "ADVOCATE",
      "userCount": 3,
      "assignedUserNames": ["John Doe", "Jane Smith", "Bob Wilson"],
      "isActive": true,
      "isSystem": false
    },
    ...
  ]
}
```

Frontend: Show a table/grid of roles with name, user count, status. Click a role to manage its permissions.

View Role Permissions + Available Ceiling

GET /api/v1/firm/roles/{roleId}/permissions

```

Response: {
  "data": {
    "roleId": "...",
    "roleName": "ADVOCATE",
    "roleCode": "ADVOCATE",
    "currentPermissions": [
      { "id": "...", "code": "CASE_MANAGEMENT:VIEW", "action": "VIEW", "scope": "TENANT" },
      { "id": "...", "code": "CASE_MANAGEMENT:CREATE", "action": "CREATE", "scope": "TENANT" }
    ],
    "availablePermissions": [
      { "id": "...", "code": "CASE_MANAGEMENT:VIEW", "action": "VIEW", "moduleCode": "CASE_MANAGEMENT",
"assigned": true },
      { "id": "...", "code": "CASE_MANAGEMENT:CREATE", "action": "CREATE", "moduleCode": "CASE_MANAGEMENT",
"assigned": true },
      { "id": "...", "code": "CASE_MANAGEMENT:EDIT", "action": "EDIT", "moduleCode": "CASE_MANAGEMENT",
"assigned": false },
      { "id": "...", "code": "BILLING:VIEW", "action": "VIEW", "moduleCode": "BILLING", "assigned": false
    },
      ...
    ]
  }
}

```

Frontend:

- `currentPermissions` = what the role currently has (show as checked checkboxes)
- `availablePermissions` = what CAN be assigned (the ceiling). Items with `assigned: true` are already assigned.
- Group by `moduleCode` for a nice UI (accordion or tabs per module)
- Show only `availablePermissions` as the checkbox options — these ARE the ceiling

Update Role Permissions

```

PUT /api/v1/firm/roles/{roleId}/permissions
Body: {
  "data": {
    "roleId": "<roleId>",
    "permissionIds": [
      "<PERM_ID_1>",
      "<PERM_ID_2>",
      ...
    ]
  }
}

Response: {
  "data": {
    "roleId": "...",
    "roleName": "ADVOCATE",
    "roleCode": "ADVOCATE",
    "permissions": [
      { "id": "...", "code": "CASE_MANAGEMENT:VIEW", ... },
      ...
    ]
  }
}

```

Frontend:

- Collect all checked permission IDs from the checkbox UI
- Send the FULL list (not add/remove — it's a REPLACE operation)
- On success, show a toast: "Permissions updated. Affected users must re-login."
- If 403 → "Permission exceeds the ceiling" → show which permission failed

Create Custom Role

```
POST /api/v1/firm/roles
Body: { "data": { "name": "Junior Advocate", "code": "JUNIOR_ADVOCATE", "description": "..." } }

Response: { "data": { "id": "...", "name": "Junior Advocate", "code": "JUNIOR_ADVOCATE", ... } }
```

Then assign permissions via the PUT endpoint above.

Delete Custom Role

```
DELETE /api/v1/firm/roles/{roleId}

Response: { "success": true, "message": "Custom role deleted successfully" }
```

Cannot delete: FIRM_ADMIN, or roles with assigned users (400 error).

Toggle Role Status

```
PATCH /api/v1/firm/roles/{roleId}/toggle

Response: { "data": { "id": "...", "isActive": false, ... } }
```

View Role Users

```
GET /api/v1/firm/roles/{roleId}/users

Response: {
  "data": [
    { "id": "...", "fullName": "John Doe", "username": "johndoe", "email": "john@test.com", "isActive": true },
    ...
  ]
}
```

2.4 Employee Permission Check (Frontend Guards)

After login, the JWT contains the user's permissions. Use them for UI guards:

```
// Example: Check if user has a permission
function hasPermission(permissionCode) {
  // Parse JWT payload (or store permissions from login response)
  const permissions = user.permissions; // Array of permission codes
  return permissions.includes(permissionCode);
}

// Example: Show/hide UI elements
{hasPermission('CASE_MANAGEMENT:CREATE') && (
  <button onClick={createCase}>Create Case</button>
)}
```



```
// Example: Route guards
const canAccessBilling = hasPermission('BILLING:VIEW');
if (!canAccessBilling) {
  return <AccessDenied />;
}
```

Permission codes to check:

- CASE_MANAGEMENT:VIEW — Can view cases
- CASE_MANAGEMENT:CREATE — Can create cases
- CASE_MANAGEMENT:EDIT — Can edit cases
- CASE_MANAGEMENT:DELETE — Can delete cases
- BILLING:VIEW — Can view invoices
- BILLING:APPROVE — Can approve invoices
- USER_MANAGEMENT:VIEW — Can view users
- USER_MANAGEMENT:EDIT — Can manage users
- ROLE_MANAGEMENT:VIEW — Can view roles
- DASHBOARD_MANAGEMENT:VIEW — Can view dashboard

2.5 Complete Flow Diagram

SUPER ADMIN

1. Login → GET /super-admin/login → accessToken
2. Create Firm → POST /super-admin/firms
→ Firm created with 4 empty roles
3. Set Ceiling → PUT /super-admin/firms/{id}/roles/{firmAdminRoleId}/permissions
→ FIRM_ADMIN now has [A, B, C, D, E]
4. (Optional) Create Custom Role → POST /super-admin/firms/{id}/roles
→ Then set its permissions via PUT override



FIRM ADMIN

1. Login → POST /auth/login → accessToken
2. View Roles → GET /firm/roles
→ See FIRM_ADMIN, ADVOCATE, PARALEGAL, CLIENT, custom
3. View ADVOCATE Permissions → GET /firm/roles/{id}/permissions
→ currentPermissions: [] (empty)
→ availablePermissions: [A, B, C, D, E] (the ceiling)
4. Assign to ADVOCATE → PUT /firm/roles/{advocateId}/permissions
→ Send [A, B, C] – within ceiling ✅
5. Try to assign SCRAPER:VIEW → 403 ❌ (not in ceiling)
6. (Optional) Create Custom Role → POST /firm/roles
→ Then assign permissions via PUT



EMPLOYEE (ADVOCATE)

1. Login → POST /auth/login → accessToken
2. JWT contains permissions: [A, B, C]
3. Frontend checks: hasPermission('A') → true
4. Backend checks: PermissionEvaluator.require('A') → OK
5. Backend checks: PermissionEvaluator.require('D') → 403

2.6 Error Handling

Status	Meaning	Frontend Action
200	Success	Process response
400	Bad request / validation error	Show error message from <code>message</code> field
401	Unauthorized / token expired	Redirect to login
401 (permissions changed)	Permission version mismatch	Force re-login with message
403	Forbidden (no permission)	Show "Access Denied" or hide UI element
404	Resource not found	Show "Not Found" message
409	Duplicate resource	Show "Already exists" message

2.7 Key Implementation Notes

1. **Permission IDs are UUIDs** — always use IDs, not codes, when assigning permissions
2. **Replace, not toggle** — PUT replaces the ENTIRE permission set. Send all checked permissions.
3. **Token refresh** — use the refresh endpoint, not re-login
4. **Permission caching** — permissions are in JWT + in-memory cache. Changes take effect immediately (permissionVersion bump forces re-login)
5. **FIRM_ADMIN cannot be deleted/toggled** by Firm Admin — only SA can manage Firm Admin
6. **Custom roles** — created by Firm Admin or SA. No ceiling at creation. Permissions assigned afterward.
7. **GLOBAL scope permissions** — never assignable to firm roles. SA-only.
8. **Firm code in login** — Firm Admin login requires `lawFirmCode` in the request body